# Unidad Temática N°3

~ Estructuras de datos ~

"Si ha elegido las estructuras de datos correctas y las cosas están bien organizadas, los algoritmos casi siempre serán evidentes.".

~ Rob Pike

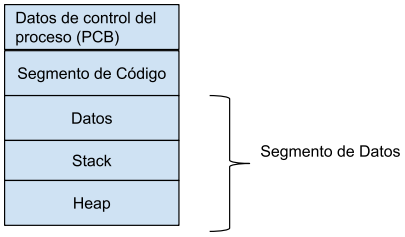
# Estructuras de datos

En esta unidad vamos a estudiar el manejo de estructuras de datos dinámicas.

**¿Por qué dinámicas?** Porque a diferencia de los vectores y matrices cuyo tamaño es fijo y no varía durante la ejecución del programa, las estructuras dinámicas pueden crecer o reducirse a medida que se ejecuta el programa.

Antes de empezar a analizar cada una de las estructuras, vamos a aclarar algunos conceptos sobre la memoria.

Cuando un proceso es ejecutado, el mismo se divide en 2 grandes segmentos: código y datos. A su vez, el segmento de datos se subdivide en: datos variables, stack y heap.




- Código: contiene el código ejecutable del programa
- Datos: contiene variables locales globales y estáticas.
- Stack: utilizado para “apilar” las llamadas a funciones y sus parámetros.
- Heap: contiene variables cuya reserva de memoria es dinámica. Este segmento puede crecer o disminuir su tamaño.

El *stack* (o pila) es la porción de memoria que el sistema operativo utiliza para almacenar las llamadas a función y crear las variables locales de la función y sus parámetros. Es una porción de memoria que el sistema operativo maneja con un algoritmo tipo LIFO (último entrado, primero en salir) por lo que decimos que “apila” las llamadas a funciones. Cuando invocamos a una función, la misma (junto con sus parámetros y variables locales) se crean en el stack. Cuando la función retorna, todas las variables creadas en el stack son destruidas liberando el espacio utilizado.

Las estructuras dinámicas estarán alojadas en el segmento denominado *heap*. Este segmento es parte de la memoria RAM y pertenece al espacio de memoria asignado al proceso que se está ejecutando. A diferencia del *stack* no hay ningún algoritmo para almacenar los datos y la responsabilidad por la reserva y liberación de memoria se delega al programador.

La descripción anterior, refiere al comportamiento de una aplicación desarrollada utilizando lenguajes como C o C++ donde es posible utilizar funciones (brindadas por el lenguaje) que permiten manipular la memoria del proceso a bajo nivel.

En lenguajes de alto nivel como Java, Pyton, Javascript esto no es necesario ya que existen otros mecanismos automáticos que gestionan el uso de la memoria del proceso.

Las estructuras dinámicas que estudiaremos son: pilas, colas y listas.

## Tipo de dato

Si bien, estas estructuras pueden almacenar cualquier tipo de dato, generalmente crearemos una clase a la que llamaremos *Nodo*.

La forma más genérica de definir esta clase es la siguiente:

```
public class MyNode<T>
{
    private T _value;
    private MyNode<T>? _next;

    public MyNode(T value)
    {
        _value = value;
        _next = null;
    }

    public T Value
    {
        get { return _value; }
    }

    public MyNode<T>? Next
    {
        get { return _next; }
    }
}
```

*Ejemplo 43.1*

Nótese el uso del tipo de dato genérico (generics) que nos permite construir nodos que almacenan valores de tipo number, string u objetos de otro tipo.

## Pilas

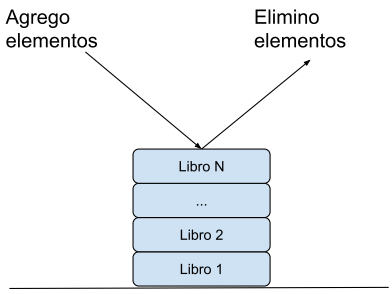
Son estructuras de datos que almacenan y recuperan sus elementos siguiendo un orden estricto. Este orden es *LIFO,* es decir, último en entrar primero en salir.

| | |
|---|---|
| Definición | es una colección de elementos de un mismo tipo de dato a los que sólo se puede acceder (introducir y eliminar) por un único lugar denominado cabeza de pila. |

En general, el uso de este tipo de estructura es adecuado cuando necesitamos invertir el orden de un conjunto de elementos o bien conocer el orden en que se fueron sucediendo determinados eventos, como por ejemplo el historial de navegación de páginas web, el historial de acciones de un programa para luego poder “deshacerlas”, etc.

Es importante destacar que la lectura sobre este tipo de estructura de datos es destructiva, es decir, para leer un elemento debemos removerlo de la pila.

Imaginemos una pila de platos o libros, el primer plato que agrego a la pila será el último que pueda extraer de la misma.




En este ejemplo, el primer libro agregado a la pila es el “Libro 1”, el segundo es el “Libro 2”, y así sucesivamente hasta el “Libro N”.

Cuando necesitemos leer los elementos de la pila, lo haremos en orden inverso, primero vamos a leer el “Libro N” y por último el “Libro 1”.

### Definición de la clase pila

```
using System;

public class MyNode<T>
{
    public T Value { get; }
    public MyNode<T> Next { get; set; }

    public MyNode(T value)
```

```
    {
        Value = value;
        Next = null;
    }
}

public class Stack<T>
{
    private MyNode<T> head;

    public Stack()
    {
        head = null;
    }

    public void Push(T value)
    {
        var newNode = new MyNode<T>(value);
        newNode.Next = head;
        head = newNode;
    }

    public T Pop()
    {
        if (IsEmpty())
        {
            return default; // devuelve null para referencias o valor por defecto
para tipos primitivos
        }

        T value = head.Value;
        head = head.Next;
        return value;
    }

    public bool IsEmpty()
    {
        return head == null;
    }
}
```

*Ejemplo 43.2*

## Operaciones sobre una pila

### create (crear):

Para crear una pila no hay que realizar ninguna acción más que inicializar la cabecera de la pila, por lo que sólo debemos invocar al constructor de la clase.

### isEmpty (está vacía)

Esta operación nos permite saber si la pila está vacía o no. Al igual que la operación de creación, es muy simple ya que sólo debemos verificar si la cabecera de la pila es *undefined*.

### push (insertar)

La operación *push* nos permite insertar un elemento nuevo a la pila. Recordemos que la estructura de datos pila siempre inserta elementos en el tope de la pila, es decir en la cabecera.

### pop (eliminar)

La operación *pop* nos permite eliminar un elemento de la pila. Recordemos que la estructura de datos pila siempre elimina elementos del tope de la pila, es decir de la cabecera.

Cabe aclarar que las operaciones de lectura sobre una pila son destructivas, en el sentido que para leer los elementos de la pila, debemos removerlos de la misma utilizando la función pop.

Tener en cuenta que antes de invocar a la función *pop* debemos verificar que la pila tenga elementos ya que de lo contrario producirá un error.

### Ejemplo

```
using System;

class Program
{
    static void Main()
    {
        // Crear una pila que almacena números
        Stack<int> p = new Stack<int>();

        // Verificar estado de la pila. Salida esperada: true
        Console.WriteLine(p.IsEmpty());

        // Agregar elementos a la pila
        p.Push(1);
        p.Push(2);
        p.Push(4);
```

```
        // Imprimir el contenido de la pila. Salida esperada: 4, 2, 1
        while (!p.IsEmpty())
        {
            Console.WriteLine(p.Pop());
        }

        // Verificar estado de la pila. Salida esperada: true
        Console.WriteLine(p.IsEmpty());
    }
}
```

*Ejemplo 43.3*

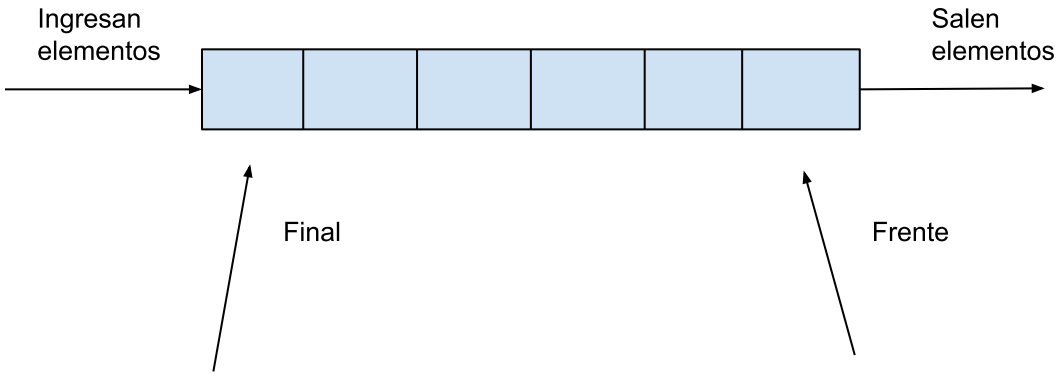
## Colas

Al igual que las pilas, son estructuras de datos que almacenan y recuperan sus elementos siguiendo un orden estricto, pero a diferencia de éstas, el orden es *FIFO,* es decir, primero en entrar primero en salir.

En cierto modo, las *colas* son parecidas a las *pilas* en el sentido de que son una colección de elementos que se insertan por un extremo (parte final), pero la diferencia con estas últimas es que la extracción de dichos elementos no se realiza por el mismo extremo por el cual se insertaron sino que se realiza por el frente (parte inicial o cabecera).

Al igual que las pilas, la operación de lectura de los elementos es una operación destructiva ya que el elemento es removido de la cola.

Imaginemos una fila de personas que esperan para ser atendidas por el cajero de un banco o una lista de trabajos que esperan su turno para hacer uso del procesador.



## Definición de la clase cola

```
using System;

public class MyNode<T>
{
    public T Value { get; }
    public MyNode<T> Next { get; set; }

    public MyNode(T value)
    {
        Value = value;
        Next = null;
    }
}

public class Queue<T>
{
    private MyNode<T> first;
    private MyNode<T> last;

    public Queue()
    {
        first = null;
        last = null;
    }

    public void Enqueue(T value)
    {
        var newNode = new MyNode<T>(value);

        if (IsEmpty())
        {
            first = newNode;
        }
        else
        {
            last.Next = newNode;
        }

        last = newNode;
    }

    public T Dequeue()
    {
        if (IsEmpty())
        {
            return default; // null para referencias, valor por defecto para tipos
primitivos
```

```
        }

        T value = first.Value;
        first = first.Next;

        if (first == null)
        {
            last = null;
        }

        return value;
    }

    public bool IsEmpty()
    {
        return first == null && last == null;
    }
}
```

*Ejemplo 43.4*

## Operaciones sobre una cola

### create (crear):

Para crear una pila no hay que realizar ninguna acción más que inicializar la los punteros de inicio y fin de la cola, por lo que sólo debemos invocar al constructor de la clase.

Para manejar una cola, utilizaremos una estructura que contenga ambos punteros, cabecera y final.

### enqueue (agregar)

La operación *enqueue* nos permite agregar un elemento nuevo a la cola. Recordemos que la estructura de datos cola siempre inserta elementos en el final de la cola.

### dequeue (eliminar)

La operación *dequeue* nos permite eliminar un elemento de la cola. Recordemos que la estructura de datos cola siempre elimina elementos del frente de la cola.

Cabe aclarar que las operaciones de lectura sobre una cola son destructivas, en el sentido que para leer los elementos de la cola, debemos removerlos de la misma utilizando la función remove.

Tener en cuenta que antes de invocar a la función *remove* debemos verificar que la cola tenga elementos ya que de lo contrario producirá un error.

### isEmpty (está vacía)

Esta operación nos permite saber si la cola está vacía o no. Al igual que la operación de creación, es muy simple ya que sólo debemos verificar si los punteros de la cola son *undefined*.

### Ejemplo

```
using System;
class Program
{
    static void Main()
    {
        // Crear una cola que almacena números
        Queue<int> q = new Queue<int>();

        // Verificar que esté vacía. Salida esperada: true
        Console.WriteLine(q.IsEmpty());

        // Encolar elementos
        q.Enqueue(1);
        q.Enqueue(2);
        q.Enqueue(4);

        // Imprimir el contenido de la cola. Salida esperada: 1, 2, 4
        while (!q.IsEmpty())
        {
            Console.WriteLine(q.Dequeue());
        }

        // Verificar que esté vacía. Salida esperada: true
        Console.WriteLine(q.IsEmpty());
    }
}
```

*Ejemplo 43.5*

## Listas

Una lista enlazada es una estructura de datos que maneja una colección de elementos dispuestos uno detrás de otro en la que cada elemento se conecta al siguiente mediante un puntero.

A diferencia de las *pilas* y *colas* no posee una política definida para el manejo de sus elementos, motivo por el cual una lista puede ser recorrida en cualquier orden, se puede insertar elementos en cualquier parte de la misma y las operaciones de lectura no son destructivas.

Al igual que con las otras estructuras de datos con las que estuvimos trabajando, para trabajar con listas debemos definir la estructura del nodo, que consta de datos y como mínimo un puntero al siguiente nodo.

Antes de comenzar a analizar las operaciones, hay que destacar que a diferencia de las estructuras vistas anteriormente, las operaciones que insertan un nodo en la lista también lo devuelven. Esta forma de trabajar nos va a permitir desarrollar estructuras más complejas como *listas de listas*.

### Definición de la clase lista

```
using System;

public class MyNode<T>
{
    public T Value { get; set; }
    public MyNode<T> Next { get; set; }

    public MyNode(T value)
    {
        Value = value;
        Next = null;
    }
}

public class List<T> where T : IComparable<T>
{
    private MyNode<T> head;

    public List()
    {
```

```
        head = null;
    }

    public MyNode<T> Push(T value)
    {
        var node = new MyNode<T>(value);

        if (head == null)
        {
            head = node;
        }
        else
        {
            var headAux = head;
            while (headAux.Next != null)
            {
                headAux = headAux.Next;
            }
            headAux.Next = node;
        }

        return node;
    }

    public T Pop()
    {
        if (head == null) return default;

        MyNode<T> headAux = head;
        MyNode<T> previous = null;

        while (headAux.Next != null)
        {
            previous = headAux;
            headAux = headAux.Next;
        }

        if (previous == null)
        {
            head = null;
        }
        else
        {
            previous.Next = null;
        }

        return headAux.Value;
    }

    public MyNode<T> InsertFirst(T value)
    {
        var node = new MyNode<T>(value);
```

```
        node.Next = head;
        head = node;
        return node;
    }

    public T RemoveFirst()
    {
        if (head == null) return default;

        T value = head.Value;
        head = head.Next;
        return value;
    }

    public MyNode<T> InsertOrdered(T value)
    {
        var node = new MyNode<T>(value);
        MyNode<T> headAux = head;
        MyNode<T> previous = null;

        while (headAux != null && headAux.Value.CompareTo(value) < 0)
        {
            previous = headAux;
            headAux = headAux.Next;
        }

        if (previous == null)
        {
            head = node;
        }
        else
        {
            previous.Next = node;
        }

        node.Next = headAux;
        return node;
    }

    public MyNode<T> InsertUnique(T value)
    {
        var node = Search(value);
        if (node == null)
        {
            return InsertOrdered(value);
        }
        return node;
    }

    public T Delete(T value)
    {
        MyNode<T> headAux = head;
```

```
        MyNode<T> previous = null;

        while (headAux != null && !headAux.Value.Equals(value))
        {
            previous = headAux;
            headAux = headAux.Next;
        }

        if (headAux == null) return default;

        if (previous == null)
        {
            head = headAux.Next;
        }
        else
        {
            previous.Next = headAux.Next;
        }

        return headAux.Value;
    }

    public MyNode<T> Search(T value)
    {
        MyNode<T> headAux = head;
        while (headAux != null && !headAux.Value.Equals(value))
        {
            headAux = headAux.Next;
        }
        return headAux;
    }

    public void Sort()
    {
        var listAux = new List<T>();
        while (head != null)
        {
            T value = RemoveFirst();
            listAux.InsertOrdered(value);
        }
        head = listAux.head;
        listAux.Clear();
    }

    public void Clear()
    {
        while (head != null)
        {
            RemoveFirst();
        }
    }
}
```

*Ejemplo 43.6*

## Operaciones sobre una lista

### Create (crear):

Para crear una lista no hay que realizar ninguna acción más que inicializar la cabecera de la lista, por lo que sólo debemos invocar al constructor de la clase.

### InsertFirst (inserta adelante)

Esta operación agrega un nodo delante del primer nodo.

### push (inserta al final)

Esta operación agrega un nodo al final de la lista.

En esta función debemos destacar la utilización del puntero auxiliar *headAux*. Este puntero es utilizado para no perder la referencia al primer nodo de la lista.

### InsertOrdered (inserta ordenado)

Esta operación permite insertar un nodo conservando el orden, en otras palabras, esta función permite insertar un nodo respetando el orden de los datos.

### Search (buscar un nodo)

Esta función permite buscar y devolver el nodo que contenga el valor pasado por parámetro.

### RemoveFirst (elimina el primer nodo)

Esta operación permite eliminar el primer nodo de la lista (similar al manejo de pilas)

### Pop (elimina el último nodo)

Esta operación permite eliminar el último nodo de la lista

### Delete (elimina un nodo particular)

Esta función permite eliminar el nodo que contiene el valor pasado por parámetro.

### Clear (vaciar la lista)

Esta operación permite vaciar la lista

### SortList (ordenar una lista)

Esta función permite ordenar una lista. Para llevar a cabo esta tarea vamos a reutilizar las funciones *RemoveFirst* e *InsertOrdered*.

### InsertUnique (inserta sin repetición)

Esta función permite insertar un nodo siempre y cuando el mismo no exista previamente en la lista. En caso de no existir lo inserta respetando el orden de la lista. Devuelve el puntero al nodo encontrado o insertado.

# Bibliografía


- Luis Joyanes Aguilar, Ignacio Zahonero Martínez. Programación en C. Segunda Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA. S.A.U., 2005. ISBN-84: 481-9844-1.
- Brian W. Kernighan, Rob Pike. La práctica de la programación. Pearson Educación. Méjico (2000)
- Debugging with CodeBlocks: http://wiki.codeblocks.org/index.php?title=Debugging_with_Code::Blocks
- Bibliotecas.
    - https://es.wikipedia.org/wiki/Biblioteca_(inform%C3%A1tica)
- Enlace dinámico
    - https://es.wikipedia.org/wiki/Enlace_din%C3%A1mico
- Enlace estático
    - https://es.wikipedia.org/wiki/Enlace_est%C3%A1tico